Web Performance Tools

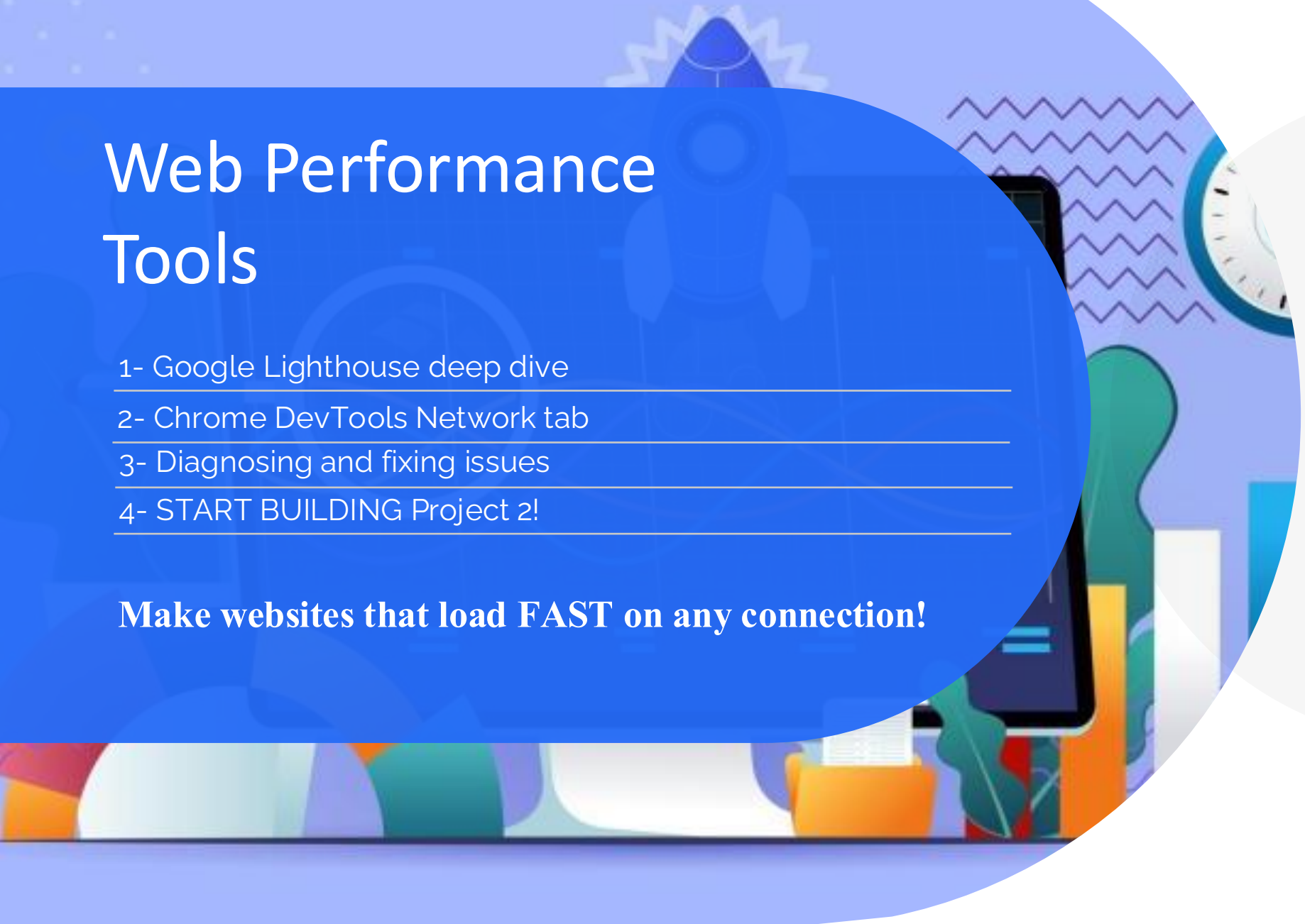
1- Google Lighthouse deep dive

2- Chrome DevTools Network tab

3- Diagnosing and fixing issues

4- START BUILDING Project 2!

Make websites that load FAST on any connection!



SCROLL

Meet Lighthouse - Your Performance Auditor

Google's Free Performance Audit Tool

What is Lighthouse? Built directly into Chrome DevTools - no installation needed!

What Lighthouse Tests:

- **Performance** - Speed, load times, optimization
- **Accessibility** - Screen readers, keyboard navigation, contrast
- **Best Practices** - Security, modern standards, console errors
- **SEO** - Search engine optimization, meta tags
- **PWA** - Progressive Web App features (ignore this for now)

Scoring System:

- **90-100 (Green)** - Excellent, professional quality
- **50-89 (Yellow)** - Needs improvement
- **0-49 (Red)** - Critical issues, fix immediately



Why Lighthouse is Amazing: It doesn't just tell you your score - it tells you exactly what's wrong and how to fix it!

Running Lighthouse - Step by Step

How to Run a Lighthouse Audit

Step-by-Step Process:

1. Open Chrome DevTools
- Press F12 (Windows/Linux) or Cmd+Option+I (Mac)
- Or right-click anywhere on page → Inspect

2. Find the Lighthouse Tab
- Look across the top tabs in DevTools
- Click "Lighthouse" (may be under >> if screen is narrow)

3. Select Categories to Test
- Check: Performance, Accessibility, Best Practices, SEO
- Uncheck: Progressive Web App (not needed for this class)

4. Choose Device Type
- Select "Mobile" (always test mobile first!)
- Desktop testing comes later

5. Click "Analyze Page Load"
- Blue button at bottom right
- Wait 30-60 seconds while Lighthouse runs tests

6. Review Your Report
- Scores appear with color coding
- Scroll down for detailed recommendations

Understanding Lighthouse Scores and Metrics

Key Performance Metrics Lighthouse Measures

FCP - First Contentful Paint

- When the first text or image appears on screen
- User sees something is happening
- **Good:** Under 1.8 seconds | **Poor:** Over 3 seconds

LCP - Largest Contentful Paint

- When the main content becomes visible
- Usually your hero image or headline
- **Good:** Under 2.5 seconds | **Poor:** Over 4 seconds

TBT - Total Blocking Time

- How long the page is unresponsive to user input
- Measures JavaScript blocking main thread
- **Good:** Under 200ms | **Poor:** Over 600ms

CLS - Cumulative Layout Shift

- How much content jumps around while loading
- Measures visual stability
- **Good:** Under 0.1 | **Poor:** Over 0.25

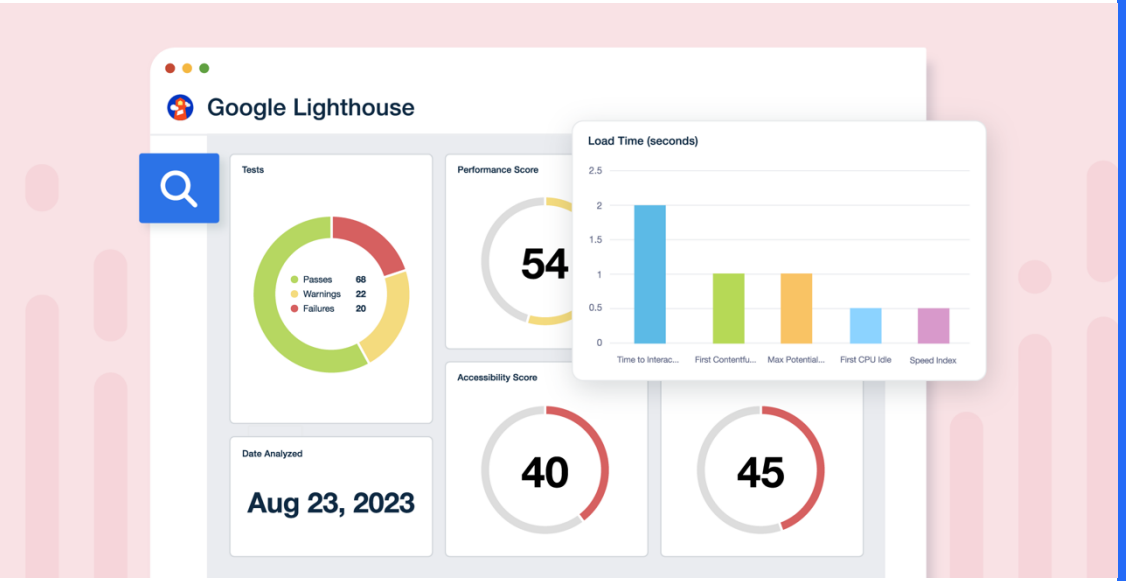
You don't need to memorize these numbers. Just understand:
Green = Good | Yellow = Needs Work | Red = Fix This

Lighthouse Recommendations

Lighthouse Tells You Exactly What to Fix!

How Recommendations Work:

- Each issue is listed as an expandable item
- Click the arrow to see details
- Shows affected files and resources
- Provides estimated time/size savings
- Includes "Learn more" link to documentation



Common Lighthouse Recommendations:

1. "Properly size images"

- Problem: Serving images larger than display size
- Fix: Use srcset or resize images

2. "Serve images in next-gen formats"

- Problem: Using JPG instead of WebP
- Fix: Convert to WebP with fallback

3. "Eliminate render-blocking resources"

- Problem: CSS/JS blocking page rendering
- Fix: Defer scripts, inline critical CSS

4. "Minify CSS/JavaScript"

- Problem: Uncompressed code files
- Fix: Remove whitespace and comments (for production)

5. "Remove unused CSS"

- Problem: Loading styles you don't use
- Fix: Delete unused rules from stylesheet

Mobile vs Desktop Testing

Always Test Both Device Types - Mobile First!



Mobile Testing:

- Simulates slower mobile processors
- Simulates slower network connections (4G)
- Smaller viewport size
- Stricter performance standards
- **Scores are typically 10-20 points lower than desktop**
- **This is normal and expected!**

Desktop Testing:

- Simulates faster processors
- Simulates faster connections
- Larger viewport
- More lenient standards
- Higher scores are typical

Your Testing Strategy:

1. Test Mobile First

- Mobile is harder to pass
- If you pass mobile, desktop is easy

2. Fix Issues for Mobile

- Optimize until mobile hits 80+

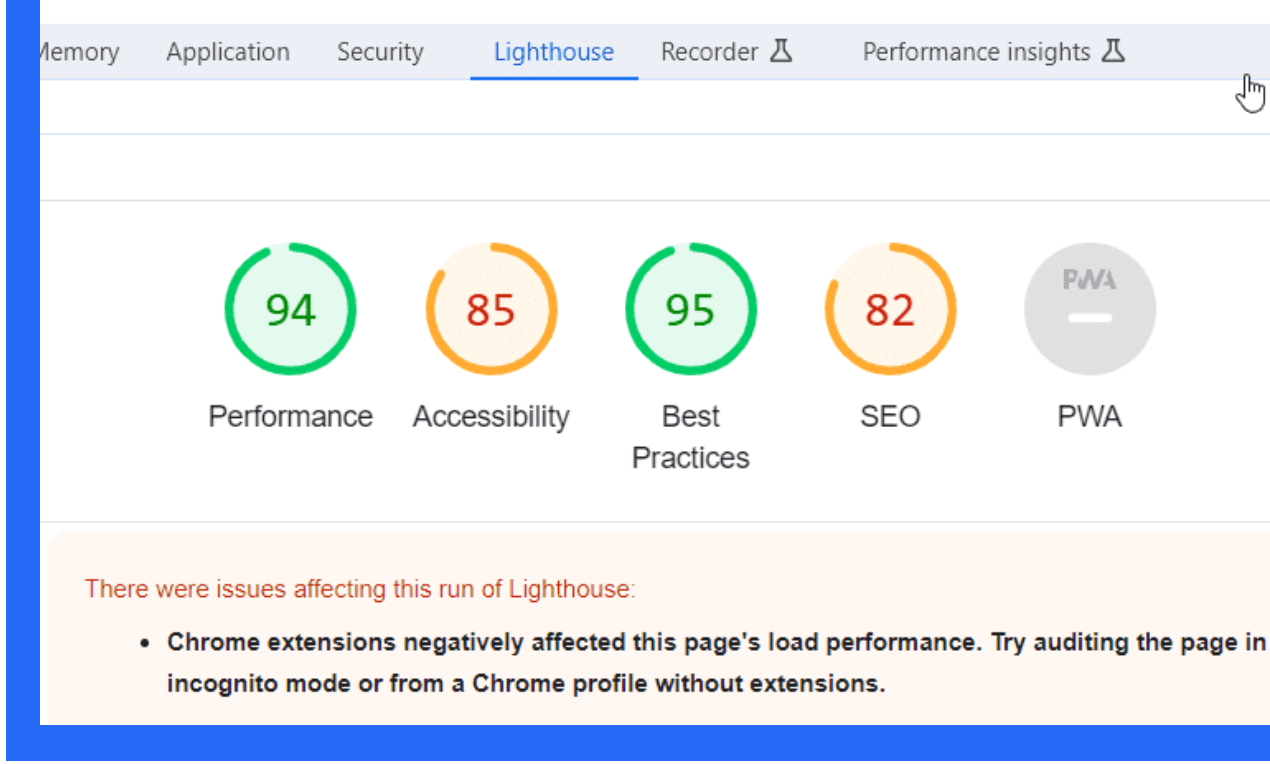
3. Then Test Desktop

- Should easily hit 90+

Lighthouse

Hands-On Practice

Wireframe = Blueprint for Your Website



Let's All Run Lighthouse Together!

Step 1: Pick any website to test
- Try a news site (CNN, New York Times)
- Try a popular brand (Nike, Apple)
- Try a social media site
- Or test one of your own previous projects

Step 2: Open Chrome DevTools (F12)
Step 3: Navigate to Lighthouse tab

Step 4: Configure settings
- Check: Performance, Accessibility, Best Practices, SEO
- Select: Mobile
- Click: "Analyze page load"

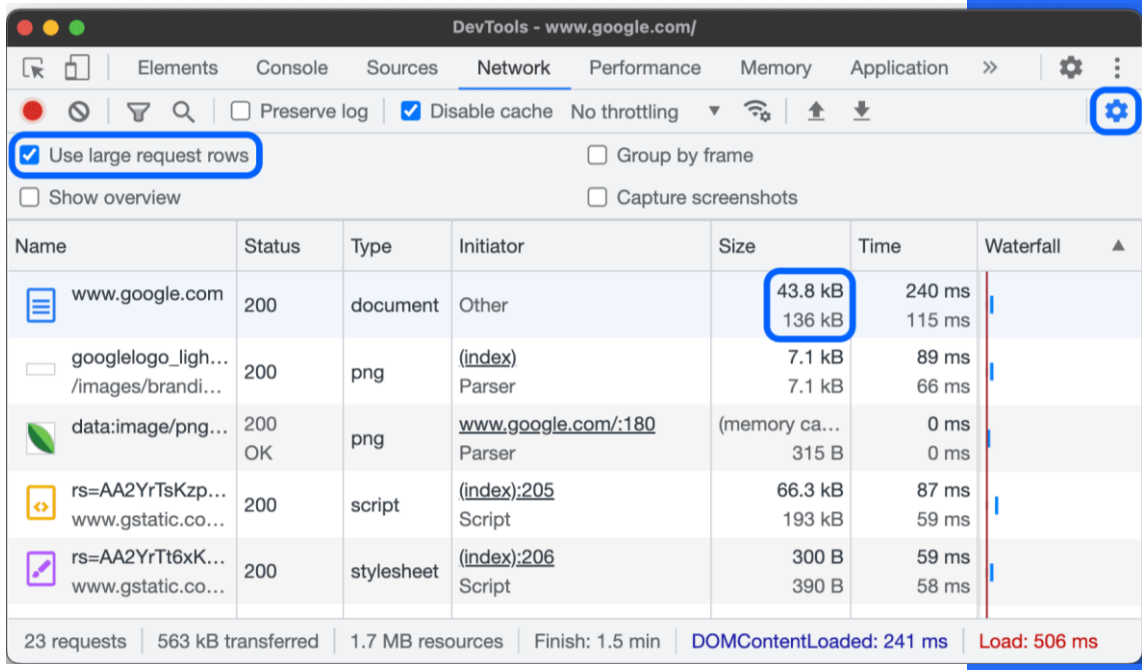
Step 5: Wait for results (30-60 seconds)
Step 6: Review scores
- What's the Performance score?
- What issues did Lighthouse find?
- Click into 1-2 recommendations to see details

Chrome DevTools Network Tab - See Everything Loading

Your Window Into Website Performance

Every Single File Loading for Your Website

- HTML documents
- CSS stylesheets
- JavaScript files
- Images (JPG, PNG, WebP, SVG)
- Fonts
- Videos
- External resources



Detailed Information for Each File:

- File name and location
- File size (how much data to download)
- Load time (how long it took)
- Status code (200 = success, 404 = not found)
- File type/content type
- When it loaded (timeline visualization)

Overall Page Statistics:

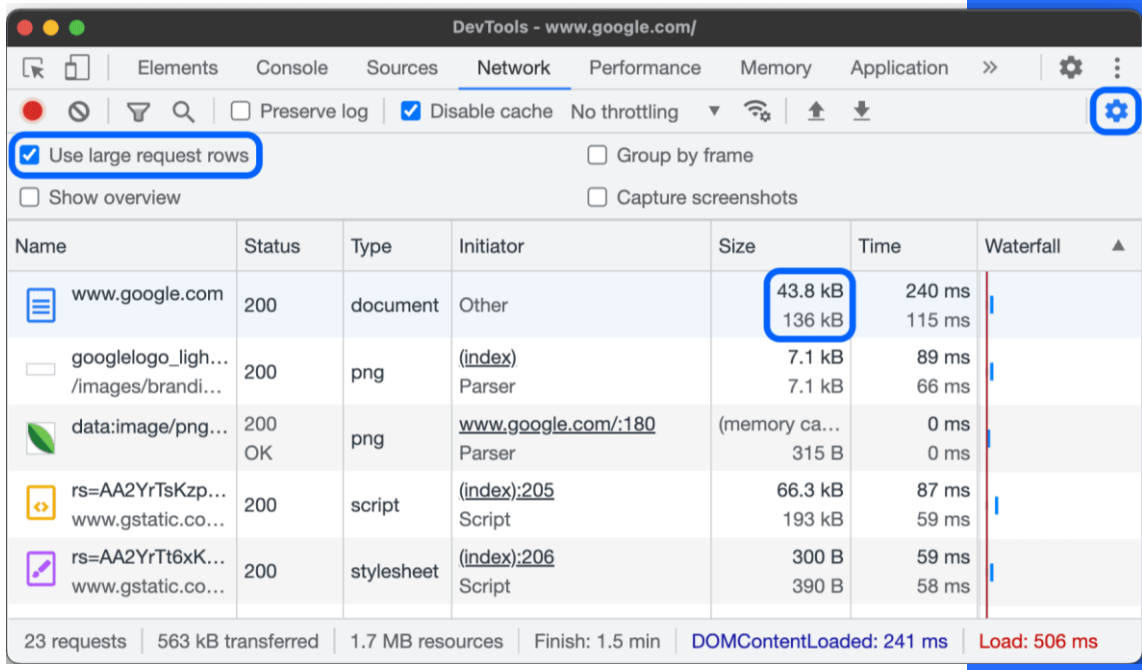
- Total number of requests
- Total page weight (all files combined)
- Total load time
- DOMContentLoaded time
- When page finished loading

Chrome DevTools Network Tab - See Everything Loading

Your Window Into Website Performance

Every Single File Loading for Your Website

- HTML documents
- CSS stylesheets
- JavaScript files
- Images (JPG, PNG, WebP, SVG)
- Fonts
- Videos
- External resources



Detailed Information for Each File:

- File name and location
- File size (how much data to download)
- Load time (how long it took)
- Status code (200 = success, 404 = not found)
- File type/content type
- When it loaded (timeline visualization)

Overall Page Statistics:

- Total number of requests
- Total page weight (all files combined)
- Total load time
- DOMContentLoaded time
- When page finished loading

Opening and Using the Network Tab

How to Use the Network Tab - Step by Step

Step 1: Open Chrome DevTools

- Press F12 (Windows/Linux) or Cmd+Option+I (Mac)
- Or right-click → Inspect

Step 2: Click the "Network" Tab

- Located across the top tabs in DevTools
- Between "Elements" and "Sources" tabs

Step 3: Refresh Your Page

- Press Cmd+R (Mac) or Ctrl+R (Windows/Linux)
- Or click the refresh button in browser
- **Network tab MUST be open before you refresh!**

Step 4: Watch Files Load in Real Time

- Each file appears as it downloads
- Rows populate the list
- Timeline fills from left to right

Bottom Status Bar Shows:

- **Requests:** Total number of files (e.g., 47 requests)
- **Transferred:** Total data downloaded (e.g., 2.3 MB)
- **Resources:** Uncompressed size (e.g., 3.1 MB)
- **Finish:** Total load time (e.g., 4.52s)
- **DOMContentLoaded:** When HTML is ready (e.g., 1.89s)
- **Load:** When everything is done (e.g., 4.52s)

Network Tab Columns - Understanding the Data

Key Information in Each Column

Name:
- Filename of the resource
- Click to see full URL path
- Can filter by searching filenames

Wireframe = Blueprint for Your Website

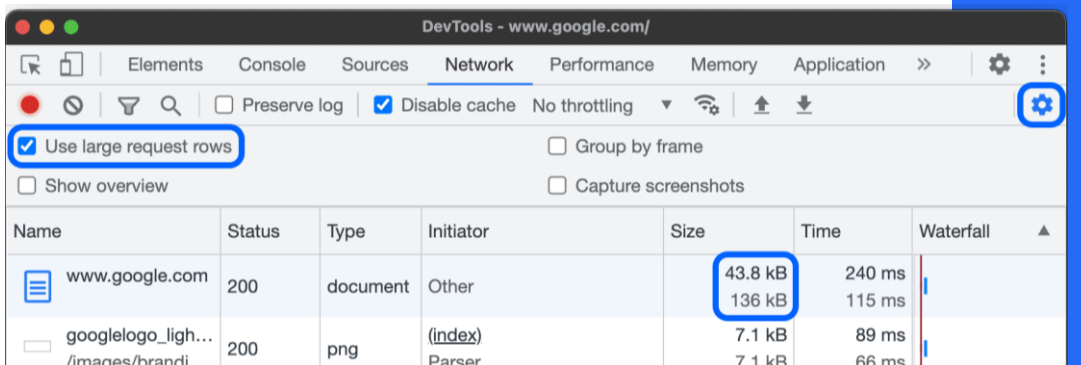
Status:
- HTTP status code
- **200** = Success (file loaded correctly)
- **304** = Not Modified (loaded from cache)
- **404** = Not Found (file missing - broken link!)
- **500** = Server Error

Type:
- File category/content type
- document (HTML)
- stylesheet (CSS)
- script (JavaScript)
- image (JPG, PNG, WebP, etc.)
- font (WOFF2, TTF)
- xhr/fetch (API requests)

Size:
- File size in KB or MB
- Shows both compressed and uncompressed
- **Smaller is better!**

Time:
- How long the file took to download
- Measured in milliseconds (ms) or seconds (s)
- **Faster is better!**

Waterfall:
- Visual timeline of when file loaded
- Shows waiting time, download time
- Helps identify bottlenecks



Network Throttling - Test Realistic Conditions

Simulate Slow Connections to Test Real User Experience

Wireframe = Blueprint for Your Website

Why Throttling Matters: You're testing on fast campus/home WiFi, but users aren't!

Throttling Options in Chrome DevTools:

No Throttling

- Your actual connection speed
- Fast WiFi or ethernet
- NOT realistic for most users
- Use only for initial development

Fast 3G (Recommended for testing)

- 1.6 Mbps download speed
- Decent mobile connection
- Realistic for many users
- **Use this for regular testing**

Slow 3G (Stress test)

- 400 Kbps download speed
- Poor connection or congested network
- Worst-case scenario
- Shows problems clearly

Offline

- No connection at all
- Tests error handling
- Not needed for this class

How to Enable Throttling:

1. Open Network tab in DevTools
2. Look for dropdown that says "No throttling"
3. Select "Fast 3G" or "Slow 3G"
4. Refresh page and watch it load slowly

Test on Fast 3G to see what real users experience!

Diagnosing Slow Files - Finding the Bottlenecks

How to Find What's Making Your Page Slow

Click on Slow-Loading Files to Investigate:

Wireframe = Blueprint for Your Website

When you click a file, you see detailed timing breakdown:

- **Queueing** - Waiting for browser to start
- **Stalled** - Waiting for available connection
- **DNS Lookup** - Finding server IP address
- **Initial Connection** - Establishing connection
- **SSL/TLS** - Secure handshake
- **Request Sent** - Sending request to server
- **Waiting (TTFB)** - Time to First Byte from server
- **Content Download** - Actually downloading the file

Network tab shows EXACTLY what's slow - no guessing needed!

Common Performance Culprits:

Huge Images (Most Common)

- Files over 500 KB
- Example: 3 MB hero image
- **Fix:** Compress, resize, convert to WebP

Too Many Requests

- 50+ files loading
- Each request adds overhead
- **Fix:** Combine files where possible

Render-Blocking Resources

- CSS/JS in head blocking page display
- Long red/orange bars at top of waterfall
- **Fix:** Defer scripts, inline critical CSS

Slow Server Response

- High "Waiting (TTFB)" time
- Problem with hosting, not your code
- **Fix:** Better hosting (for future projects)

Disable Cache for Accurate Testing

Always Disable Cache When Testing Performance

What is Browser Cache?

Wireframe ≡ Blueprint for Your Website

- Browser saves files locally after first visit
- On repeat visits, loads files from hard drive instead of downloading
- Makes repeat visits much faster
- Creates false impression of performance

The Testing Problem:

With Cache Enabled (Default):

- First visit: Downloads 2 MB, takes 4 seconds
- Second visit: Loads from cache, takes 0.5 seconds
- **You think:** "My site is fast!"
- **Reality:** Only fast for repeat visitors

With Cache Disabled:

- Every refresh simulates first-time visitor
- Downloads everything fresh
- Shows true performance
- **This is what you need to test!**

How to Disable Cache:

1. Open Network tab in DevTools
2. Find checkbox labeled "Disable cache"
3. Check the box
4. Keep DevTools open (cache only disabled while DevTools is open)
5. Now refresh and test

Important: First-time visitors are your most critical audience. They decide in seconds whether to stay or leave. Test for them!

Network Tab Hands-On Practice

Try This - Network Tab Practice

Your Task (5 minutes):
Wireframe = Blueprint for Your Website

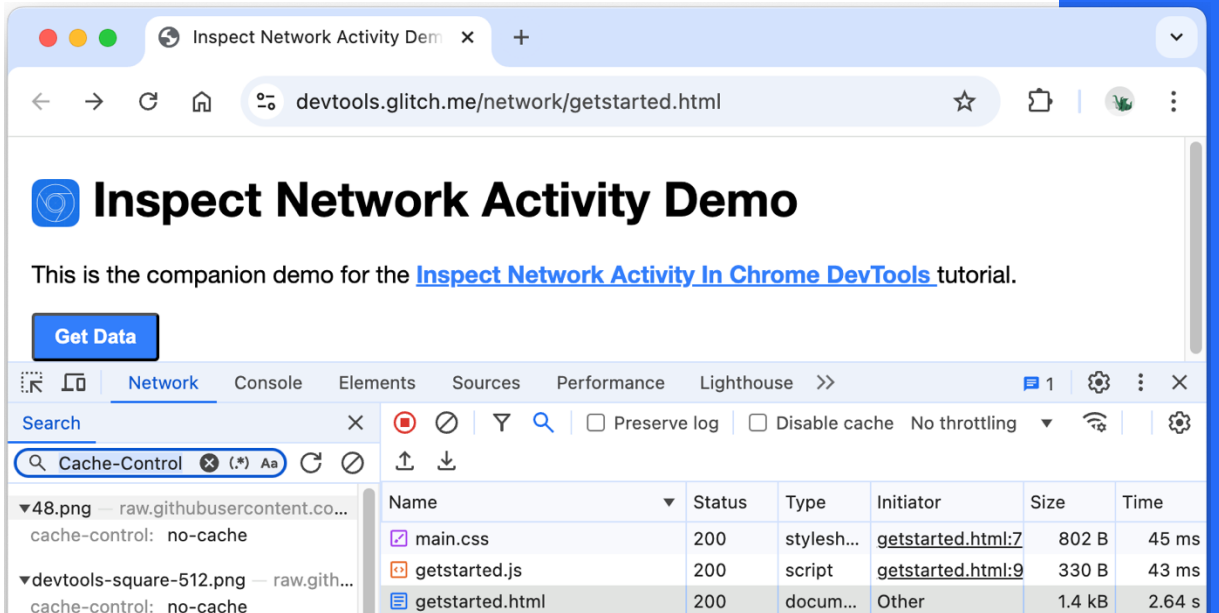
Step 1: Open any website (news site, portfolio, brand site)
Step 2: Open Chrome DevTools (F12)
Step 3: Click Network tab
Step 4: Check "Disable cache" checkbox
Step 5: Set throttling to "Fast 3G"
Step 6: Refresh the page (Cmd/Ctrl + R)
Step 7: Observe and analyze:

- How many total requests?
- What's the total page size?
- How long did it take to load?
- What's the largest file?
- What type of file is slowest?

Step 8: Click on the largest file

- Look at the Size
- Check the Time
- View the Timing breakdown

Bonus: Try throttling to "Slow 3G" and refresh again. See the dramatic difference!



Performance Budgets - Set Your Limits

What is a Performance Budget?
A performance budget is a set of limits you define BEFORE you start building to ensure your site stays fast.

Why Performance Budgets Matter:
Prevents Problem Creep:
- Easy to add "just one more image"
- Before you know it, page is 5 MB
- Budget forces conscious decisions

Sets Clear Goals:
- Know your target before building
- Test against specific numbers
- Pass/fail is objective, not subjective

Keeps You Accountable:
- Check budget after every change
- Catch problems early
- No surprises at the end

Common Budget Metrics:
- **Total Page Weight:** Under 1-2 MB
- **Number of Requests:** Under 30-50 requests
- **Load Time on 3G:** Under 3 seconds
- **Lighthouse Score:** 80+ on mobile
- **Individual Image Size:** Under 200 KB each
- **JavaScript Size:** Under 100 KB total
- **CSS Size:** Under 50 KB total

Build with limits. Test against limits. Stay within limits.



Common Performance Issues - What to Watch For



The Six Most Common Student Performance Problems

Problem 1: Huge Unoptimized Images
- Images straight from camera (3-5 MB each)
- Not compressed or resized
- Wrong format (JPG instead of WebP)

Problem 2: No Lazy Loading
- All images load immediately
- Below-fold images waste bandwidth
- Missing loading="lazy" attribute

Problem 3: Render-Blocking CSS/JavaScript
- Scripts in HTML head without defer
- CSS blocking page rendering
- User sees blank page while waiting

Problem 4: Too Many Font Weights
- Loading 6-8 font weights
- Each weight adds 100+ KB
- Only using 2-3 of them

Problem 5: No Image Dimensions
- Missing width/height attributes
- Content jumps as images load
- High Cumulative Layout Shift score

Problem 6: Testing Only on Fast WiFi
- Never throttling connection
- Site seems fast but isn't
- Reality hits when users complain

These account for 90% of performance issues in student projects. Learn to spot and fix them!

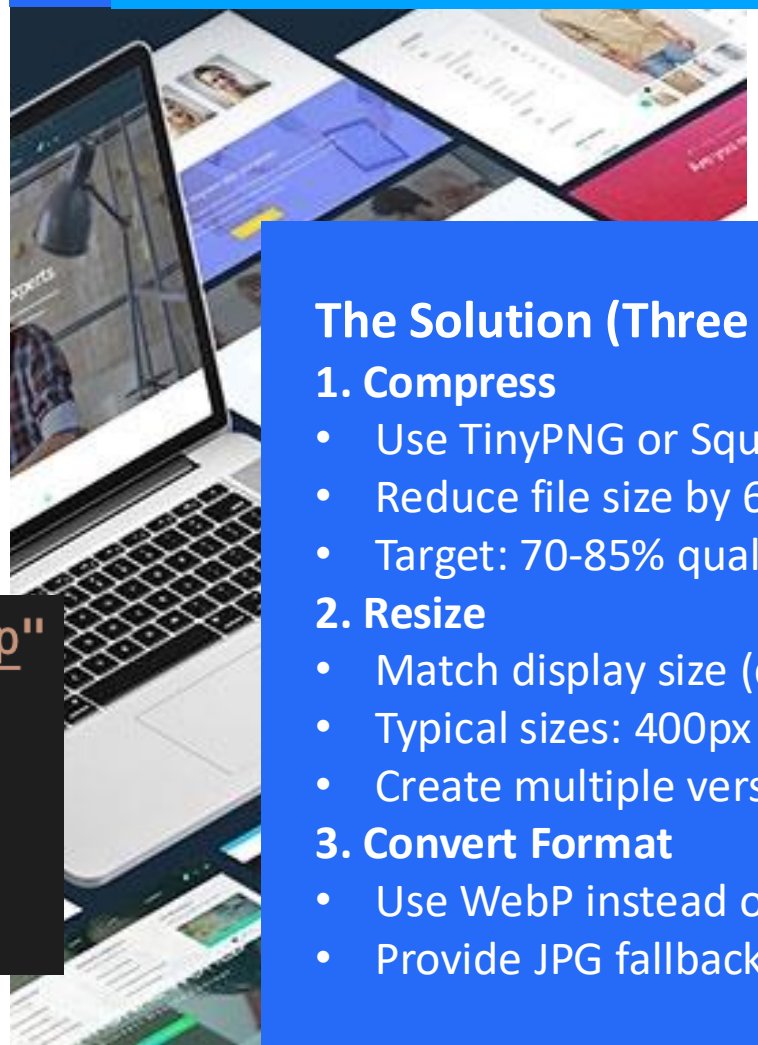
Fix #1 –
Huge Images

Why This Is a Problem:
One image takes 10+ seconds to load on mobile. Multiple images make page unusable.

Problem: Images Are Too Large

Common Scenario:

- Photo from phone: 4.2 MB, 4000 x 3000 pixels
- Downloaded stock photo: 3.8 MB, 3500 x 2500 pixels
- Screenshot: 2.1 MB, 2560 x 1440 pixels



```
<img src="hero-optimized.webp"
     width="1200"
     height="800"
     loading="lazy"
     alt="Descriptive text">
```

The Solution (Three Steps):

1. Compress

- Use TinyPNG or Squoosh
- Reduce file size by 60-80% with no visible quality loss
- Target: 70-85% quality setting

2. Resize

- Match display size (don't serve 3000px to display 800px)
- Typical sizes: 400px (mobile), 800px (tablet), 1200px (desktop)
- Create multiple versions for srcset

3. Convert Format

- Use WebP instead of JPG (25-35% smaller)
- Provide JPG fallback with picture element

Fix #2 – Missing Lazy Loading

Problem: All Images Load Immediately

What Happens Without Lazy Loading:

- Page has 15 images
- Only 3 are visible when page loads
- Browser downloads all 15 anyway
- User waits while downloading 12 unseen images
- Wastes bandwidth and time

The Rule:

- First screen content = loading="eager" or no attribute
- Everything below fold = loading="lazy"
- Browser handles everything automatically

The Solution: Add One Attribute

For Below-Fold Images:

```
<img src="article-photo.jpg"
     loading="lazy"
     width="800"
     height="600"
     alt="Article photo">
```

For Above-Fold Images (Hero, Logo):

```
<img src="hero.jpg"
     loading="eager"
     width="1200"
     height="800"
     alt="Hero image">
```

Fix #3 – Render-Blocking CSS

Problem: CSS Blocks Page Rendering

What Causes Blocking: CSS in the HTML head stops browser from displaying page until stylesheet loads.
Result: User sees blank white page while CSS downloads

Option 1: Keep CSS Small and Simple (Recommended)

- Single CSS file under 50 KB loads quickly
- Minimal blocking time
- Easiest approach for student projects

Option 2: Critical CSS Inline

```
<head>
  <style>
    /* Only essential above-fold styles */
    body { margin: 0; font-family: Arial; }
    .header { background: #333; }
  </style>
  <link rel="stylesheet" href="main.css">
</head>
```

Option 3: Load Non-Critical CSS Asynchronously

```
<!-- Print styles don't need to block -->
<link rel="stylesheet"
      href="print.css"
      media="print">
```

Fix #4 - Too Many Font Weights

Problem: Loading Unnecessary Font Weights

Common Mistake:

```
<!-- Loading 6 weights = 600+ KB -->
<link href="https://fonts.googleapis.com/css2?
family=Open+Sans:wght@300;400;500;600;700;800">
```

Each Weight Adds:

- Regular weight: 100-150 KB
- Bold weight: 100-150 KB
- Light weight: 100-150 KB
- Total for 6 weights: 600-900 KB

But You Only Use:

- Regular (400) for body text
- Bold (700) for headings

The Solution: Load Only What You Use

```
<!-- Loading 2 weights = 200-300 KB -->
<link href="https://fonts.googleapis.com/css2?
family=Open+Sans:wght@400;700">
```

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

Best Practice:

- Limit to 1-2 font families
- Limit to 2-3 weights total

Fix #5 - Layout Shift (Missing Dimensions)

Problem: Content Jumps While Page Loads

What Causes Layout Shift: Images without width and height attributes load and push content down after page initially renders.

User Experience:

- User starts reading text
- Image loads and pushes everything down
- User loses their place
- Very frustrating!
- High CLS score in Lighthouse

How It Works:

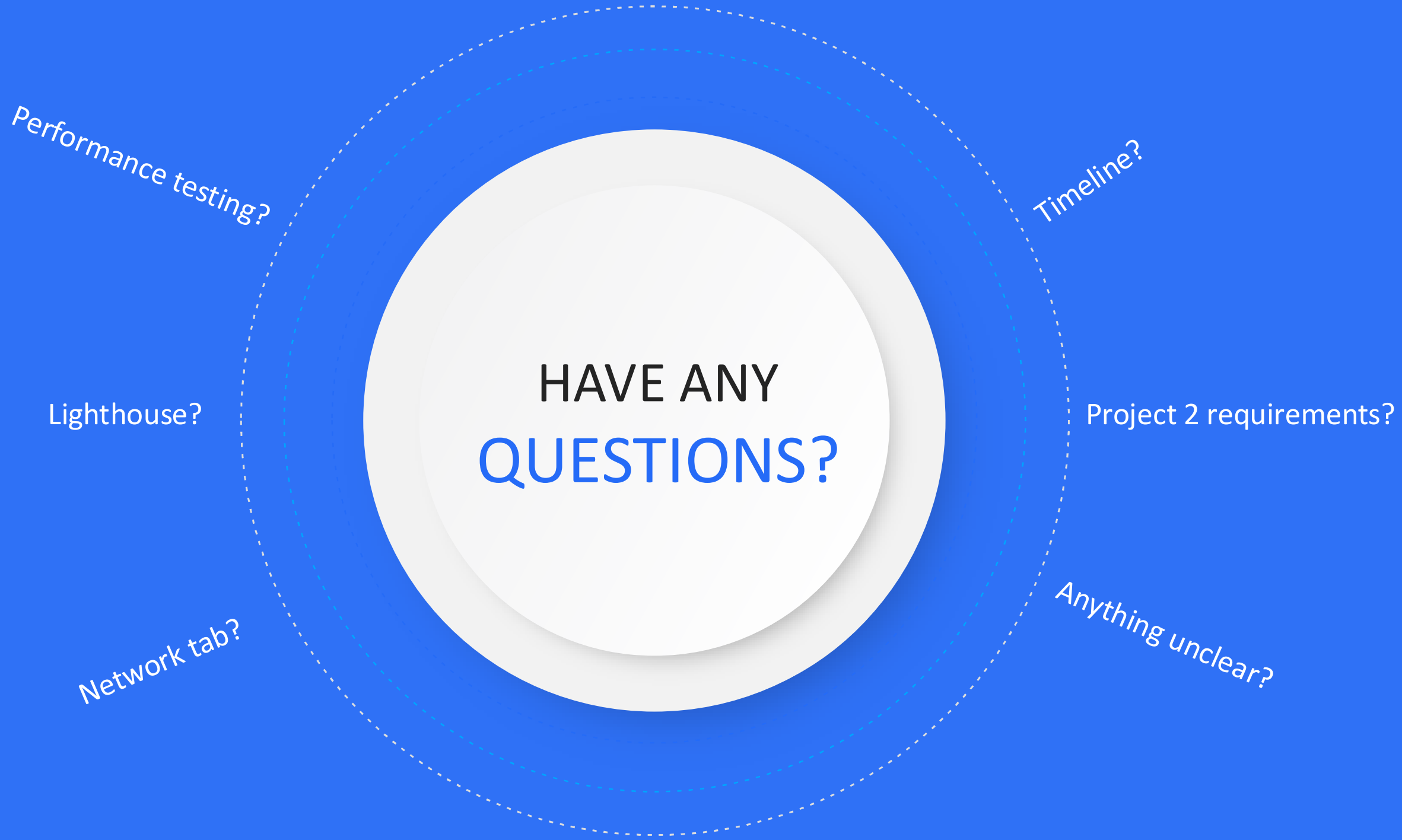
- Browser reads width and height
- Reserves that exact space in layout
- Image loads into reserved space
- No content movement

Bad Code:

```
<!-- Browser doesn't know image size -->
<img src="photo.jpg" alt="Photo">
<!-- Page renders, image loads, content shifts -->
```

Good Code:

```
<!-- Browser reserves space before image loads -->
<img src="photo.jpg" alt="Photo" width="800" height="600">
<!-- Page renders, space reserved, no content shift -->
```


Performance testing?
Timeline?
HAVE ANY
QUESTIONS?
Lighthouse?
Project 2 requirements?
Network tab?
Anything unclear?